

MQTT

一、MQTT协议介绍

MQTT (Message Queuing Telemetry Transport, 消息队列遥测传输) 是一种轻量级的、基于发布/订阅 (Publish/Subscribe) 模式的**通信协议**, 最初由 IBM 在 1999 年设计, 后来成为 **OASIS 标准**, 并广泛应用于 **物联网 (IoT)**、**移动应用**、**智能家居**、**车联网**、**工业控制**等场景。

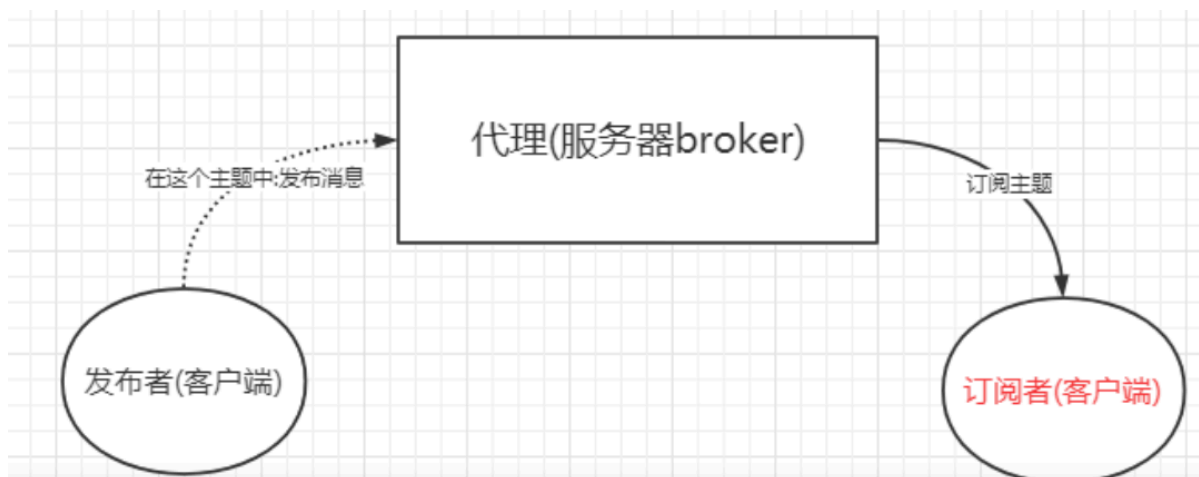
二、MQTT协议核心特点

1. **轻量级**: 协议头部非常小 (最小头部仅 2 字节), 适合带宽和资源受限的设备。
2. **低功耗**: 适用于电池供电或网络不稳定的设备, 比如传感器、嵌入式设备等。
3. **发布/订阅模式 (Pub/Sub)**: 不同于传统的客户端-服务器 (Client-Server) 点对点通信, MQTT 使用**代理 (Broker)** 作为中间人, 实现发布者和订阅者之间的解耦。
4. **可靠传输支持三种不同的服务质量等级 (QoS, Quality of Service)**, 适应不同可靠性要求:
 - QoS 0: 最多一次 (At most once), 不保证送达。
 - QoS 1: 至少一次 (At least once), 可能重复。
 - QoS 2: 恰好一次 (Exactly once), 确保只送达一次, 但开销最大。
5. **支持断线重连与遗嘱消息 (Last Will and Testament, LWT)** 客户端异常断开时, 可以发送预设的“遗嘱”消息通知其他订阅者。
6. **保持连接状态 (Keep Alive & Session Awareness)**: MQTT 支持持久会话 (Clean Session), 可以记住未接收的消息等状态信息。

三、MQTT协议架构

MQTT 是基于**客户端-代理 (Client-Broker)** 模型的:

- **客户端 (Client)**: 可以是发布者 (Publish)、订阅者 (Subscribe) 或者两者兼具。任何运行 MQTT 库的设备, 如传感器、手机、服务器等都可以是 MQTT 客户端。
- **代理 (Broker)**: 是消息的中转站, 负责接收所有客户端发来的消息, 并根据主题 (Topic) 将消息分发给订阅了该主题的客户端。常见的 Broker 有: Eclipse Mosquitto EMQX HiveMQ VerneMQ Alibaba Cloud IoT Platform (阿里云物联网平台也内置 MQTT Broker)
- 实现 MQTT 协议需要客户端和服务端通讯完成, 在通讯过程中, MQTT 协议中有三种身份: 发布者 (Publish)、代理 (Broker) (服务器)、订阅者 (Subscribe)。其中, 消息的发布者和订阅者都是客户端, 消息代理是服务器, 消息发布者可以同时是订阅者



MQTT传输的消息分为: 主题(Topic) 和 负载(payload)两部分:

1. Topic: 可以理解为消息的类型, 订阅者订阅(Subscribe)后, 就会收到该主题的消息内容(payload)
2. payload: 可以理解为消息的内容, 是指订阅者具体要使用的内容

1. MQTT客户端

一个使用MQTT协议的应用程序或者设备, 它总是建立到服务器的网络连接。

客户端可以:

1. 发布其他客户端可能会订阅的信息
2. 订阅其他客户端发布的信息
3. 退定或删除应用程序的消息
4. 断开与服务器的连接

2.MQTT服务器端

MQTT服务器以称为“消息代理”(Broker), 可以是一个应用程序或一台设备, 它是位于消息发布者和订阅者之间。

服务器端可以:

1. 接受来自客户的网络连接
2. 接受客户发布的应用信息
3. 处理来自客户端的订阅和退订请求
4. 向订阅的客户转发应用程序消息

3. 发布/订阅 主题 会话

MQTT是基于发布(Publish)订阅(Subscribe)模式来进行通信及数据交换的, 与HTTP的请求(Request)应答(Response)的模式有本质的不同

订阅者(Subscriber)会向 消息服务器(Broker)订阅一个主题(Topic)。成功订阅后, 消息服务器会将该主题下的消息转发给所有订阅者。

(1)MQTT 主题：

主题 (Topic) 是 MQTT 消息的分类标识，类似于消息队列中的“频道”或“路由键”。

主题(Topic)以“/”为分隔符区分不同的层级，包含通配符‘+’ 或 ‘#’的主题又称为主题过滤器(Topic Filters); 不含通配符的成为主题名(Topic Names)

主题是**层次化、基于斜杠分隔的字符串**，例如：

```
home/livingroom/temperature
sensor/+/humidity （使用通配符）
```

(2)MQTT主题通配符：

①、单层通配符->+

匹配主题中的一层，例如：

- 订阅 `sensor/+/temperature` ，可以匹配：`sensor/room1/temperature`或 `sensor/room2/temperature`
- 但不能匹配 `sensor/room1/humidity` 或多级如 `sensor/room1/floor1/temperature`

②、多层通配符->#

匹配主题的**多层或剩余全部层级**，必须放在主题最后，例如：

- 订阅 `sensor/#` ，可以匹配：`sensor/room1/temperature`或`sensor/room1/floor1/humidity`
- 但不能以 `#` 作为非最后一层，如 `sensor/#/temp` 是非法的。

发布者(Publisher)只能向主题名发布消息，**订阅者(Subscriber)**则可以通过订阅主题过滤器来通配多个主题名称

注意：主题是**大小写敏感**的，且支持 UTF-8 编码（但建议使用 ASCII 以保证兼容性）

(3)会话(Session):

每个客户端与服务器建立连接后就是一个会话，客户端和服务端之间有状态交互。会话存在于一个网络之间，也可能在客户端和服务端之间跨越多个连续的网络连接。

四、MQTT报文类型

MQTT 的通信基于一系列**轻量化的控制报文 (Control Packets)** ， 主要包括：

报文类型	说明
CONNECT	客户端请求连接
CONNACK	连接确认
PUBLISH	发布消息
PUBACK	QoS1 消息确认
PUBREC / PUBREL / PUBCOMP	QoS 2 相关确认（共四个报文）

报文类型	说明
SUBSCRIBE	订阅主题
SUBACK	订阅确认
UNSUBSCRIBE	取消订阅
UNSUBACK	取消订阅确认
PINGREQ	心跳请求（客户端发起）
PINGRESP	心跳响应（服务端回应）
DISCONNECT	客户端主动断开连接

五、MQTT数据包格式

在MQTT协议中，一个MQTT数据包由: 固定头(Fixed header)、可变头(Variable header)、消息体(payload)三部分构成。

MQTT数据包格式如下:



固定报头(fixed header)						可变报头		有效负载
字节1			字节2 帧剩余长度					
7-4位 报文类型		3-0位 标志位						
0	reserverd	0	0	0	0			
1	connect	0	0	0	0			
2	connack	0	0	0	0			
3	publish	DPU	QoS	QoS	RETAIN			
4	puback	0	0	0	0			
5	pubrec	0	0	0	0			
6	pubrel	0	0	0	0			
7	pubcomp	0	0	0	0			
8	subscribe	0	0	0	0			
9	suback	0	0	0	0			
10	unsubscribe	0	0	0	0			
11	unsuback	0	0	0	0			
12	pingreq	0	0	0	0			
13	pingresp	0	0	0	0			
14	disconnect	0	0	0	0			
15	reserverd	0	0	0	0			
DUP1 =控制报文的重复分发标志 QoS2 = PUBLISH报文的的服务质量等级 RETAIN3 = PUBLISH报文的保留标志								

1. **固定头(Fixed header):** 存在于所有MQTT数据包中, 表示数据包类型及数据包的分组类标识, 如连接, 发布, 订阅, 心跳等。其中固定头是必须的, 所有类型的MQTT协议中, 都必须包含固定头。
2. **可变头(Variable header):** 存在于部分MQTT数据包中, 数据包类型决定了可变头是否存在及其具体内容。可变头部不是可选的意思, 而是指这部分在有些协议类型中存在, 在有些协议中不存在。
3. **消息体 (Payload):** 存在于部分MQTT数据包中, 表示客户端收到的具体内容。与可变头一样, 在有些协议类型中有消息内容, 有些协议类型中没有消息内容。

1. 固定头(Fixed header):

固定头存在于所有MQTT数据包中。

固定头包含两部分内容：

①、首字节(1字节)

②、剩余消息报文长度(从第二个字节开始, 长度为1-4字节)

剩余长度是当前包中剩余内容长度的字节数，包括变量头和有效负载中的数据)。剩余长度不包含用来编码剩余长度的字节。

剩余长度使用了一种可变长度的结构来编码，这种结构使用单一字节表示0-127的值。大于127的值如下处理：

每个字节的低7位用来编码数据，最高位用来表示是否还有后续字节。因此每个字节可以编码128个值，再加上一个标识位。剩余长度最多可以用四个字节来表示。

bit 地址	7	6	5	4	3	2	1	0
Byte 1	MQTT数据包类型				不同类型MQTT数据包的具体标识			
Byte 2...	剩余长度							

(1) MQTT数据包类型（报文类型）：

byte1 [bit7:bit4],标识4位无符号值

报文类型	字段值	数据方向	描述	7-4bit值
保留	0	禁用	保留	0000
CONNECT	1	Client--->Server	客户端连接到服务器	0001
CONNACK	2	Server ---> Client	连接确认	0010
PUBLISH	3	Client <--> Server	发布消息	0011
PUBACK	4	Client <--> Server	发布确认(QoS1)	0100
PUBREC	5	Client <--> Server	消息已接收(QoS2 第一阶段)	0101
PUBREL	6	Client <--> Server	消息释放(QoS2 第二阶段)	0110
PUBCOMP	7	Client <--> Server	发布结束(QoS2 第三阶段)	0111
SUBSCRIBE	8	Client ---> Server	客户端订阅请求	1000
SUBACK	9	Server--->Client	服务端订阅确认	1001
UNSUBSCRIBE	10	Client ---> Server	客户端取消订阅	1010

(2) 标志位

byte1 [bit3:bit0], 表示某些报文类型的控制字段，实际上只有少数报文类型有控制位

报文类型	固定头标记	Bit 3	Bit2	BIT1	Bit 0
CONNECT	保留	0	0	0	0
CONNACK	保留	0	0	0	0
PUBLISH	Used in MQTT3.1.1(V5.0可从官网查看)	DUP	QoS	QoS	RETAIN
PUBACK	保留	0	0	0	0
PUBREC	保留	0	0	0	0
PUBREL	保留	0	0	1	0
PUBCOMP	保留	0	0	0	0
SUBSCRIBE	保留	0	0	1	0
SUBACK	保留	0	0	0	0
UNSUBSCRIBE	保留	0	0	1	0
UNSUBACK	保留	0	0	0	0
PINGREQ	保留	0	0	0	0
PINGRESP	保留	0	0	0	0

1. 其中Bit[3]为DUP字段，如果该值为1，表明这个数据包是一条重复的消息；否则该数据包就是第一次发布的消息

2. Bit[2-1]为QoS字段:

如果Bit 1 和 Bit 2都为0，表示QoS 0: 至多一次；

如果Bit 1为1，表示QoS 1: 至少一次；

如果Bit 2为1，表示QoS 2: 只有一次；

如果同时将Bit 1 和 Bit 2都设置成1，那么客户端或服务器认为这是一条非法的消息，会关闭当前连接。

3. MQTT消息QoS

MQTT发布消息服务质量保证(QoS)不是端到端的，是客户端与服务端之间的。订阅者收到MQTT消息的QoS级别，最终取决于发布消息的QoS和主题订阅的QoS

QoS: 服务质量是指客户端和服务端之间的服务质量

发布消息的QoS	主题订阅的QoS	接收消息的QoS
0	0	0
0	1	0
0	2	0
1	0	0
1	1	1
1	2	1
2	0	0
2	1	1
2	2	2

<https://bl>

2.可变头(Variable Header)

可变头的意思是可变化的消息头部。有些报文类型包含可变头部有些报文则不包含。可变头部在固定头部和消息内容之间，其内容根据报文类型不同而不同。

bit	7	6	5	4	3	2	1	0	存在的报文类型
6 Byte	Protocol name（协议名）（UTF-8字符串“MQTT”）								CONNECT
1 Byte	Protocol Version（版本号）								CONNECT
1 Byte	Connect flags（连接标记）								CONNECT
	User Name Flag	Password Flag	Will Retain	Will Qos	Will Flag	Clean Session	Reserved		
2 Byte	Keep Alive timer（心跳时长）								CONNECT
1 Byte	连接确认标识 7-1位是保留位必须设置为0，0是session present)								CONNACK
1 Byte	Connect return code（连接返回码）								CONNACK
3~32767 Byte	Topic name（主题名称）								PUBLISH
2 byte	Message ID（消息ID）								PUBLISH(Qos>0), PUBACK, PUBREC,PUBREL, PUBCOMP,SUBSCRIBE, SUBACK,UNSUBSCRIBE

协议版本

为无符号值表示客户端的版本等级。3.1.1版本

MQTT会话

MQTT客户端向服务器发起CONNECT请求时, 可以通过'Clean Session'标志设置会话。

'Clean Session'设置为0, 表示创建一个持久会话, 在客户端断开连接时, 会话仍然保持并保存离线消息, 直到会话超时注销。

'Clean Session'设置为1, 表示创建一个新的临时会话, 在客户端断开时, 会话自动销毁

Will Flag /Will QoS/Will Retain

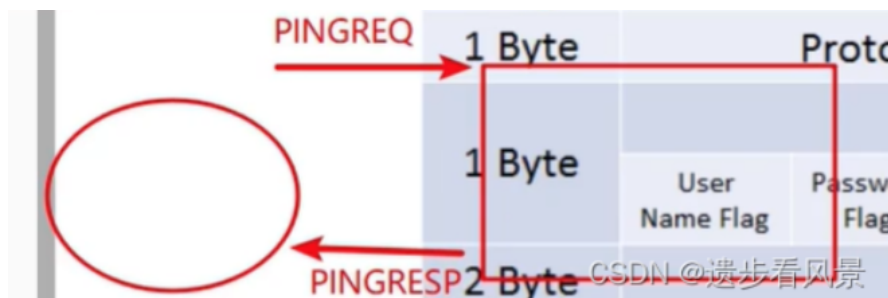
Will Flag : 遗言标志位

Will QoS: 遗言的消息质量

Will Retain: 遗言的保持状态

Keep Alive timer(心跳时长)

心跳协议:



Keep Alive timer 最大时间间隔



3. 消息体(Payload)

有些报文类型是包含Payload的, Payload的意思是消息载体的意思。如PUBLISH的Payload就是指消息内容(应用程序发布的消息内容)。而CONNECT的Payload则包含Client Identifier, Will Topic, Will Message, Username, Password等信息

包含payload的报文类型如下:

报文类型	是否需要payload
CONNECT	Required
CONNACK	None
PUBLISH	Optional
PUBACK	None
PUBREC	None
PUBREL	None
PUBCOMP	None
SUBSCRIBE	Required
SUBACK	Required
UNSUBSCRIBE	Required
UNSUBACK	None
PINGREQ	None
PINGRESP	None
DISCONNECT	None

(1) CONNECT报文类型包含的消息体内容：

1. **客户端标识符 (Client ID)** (必填) 唯一标识一个 MQTT 客户端，Broker 用它来识别和管理会话。长度一般为 1~23 字节（对于某些 Broker 可支持更长）。必须提供，否则 Broker 通常会拒绝连接或自动生成一个（视具体实现而定）。
2. **遗嘱主题 (Will Topic)** (可选，仅在 Will Flag = 1 时存在) 表示当客户端异常断开时，Broker 将发布遗嘱消息到哪个主题。
3. **遗嘱消息内容 (Will Payload)** (可选，仅在 Will Flag = 1 时存在) 遗嘱消息的实际内容 (Payload)，比如 "设备离线了"。
4. **用户名 (Username)** (可选，仅在 Username Flag = 1 时存在) 用于身份认证，通常是字符串，如 "user123"。
5. **密码 (Password)** (可选，仅在 Password Flag = 1 时存在) 与用户名配合使用，通常是二进制或字符串形式的密码。

(2) SUBSCRIBE报文类型包含的消息体内容：

1. **Payload 是一个订阅项的列表 (1个或多个)**
2. **每一个订阅项的结构为：Topic Filter (主题过滤器)：**

①、UTF-8 编码的字符串（可变长度）表示要订阅的主题名称或带通配符的过滤器，例如：

`sensor/temperature`home/+`humidity`device/#`

②、QoS（1 字节）：表示客户端希望接收该主题消息的 最大服务质量等级，取值范围为：0：最多一次 1：至少一次 2：恰好一次

六、MQTT QOS服务质量

MQTT 的 QoS（Quality of Service，服务质量等级）是该协议中一个非常核心且重要的概念，它定义了 MQTT 消息在 发布者（Publisher）与订阅者（Subscriber）之间传递时，消息的可靠传输程度

1. 什么是QOS

QoS（Quality of Service，服务质量）是 MQTT 用来控制消息从发送端到接收端的传递可靠性与顺序性的机制。

由于 MQTT 是一种基于网络传输的轻量级协议，可能会面临网络不稳定、丢包、重复、乱序等问题，因此 MQTT 提供了 三种不同的 QoS 等级，允许用户根据业务需求，在可靠性与性能之间做权衡

2. QOS等级

QOS 等级	名称 (中文)	名称(英文)	QOS 值	特点	是否保证送达	是否保证重复	适用
QOS 0	最多一次	At Most Once	0	消息发送一次，不保证到达 Broker 或 Subscriber，可能丢失	否	否	对消
QOS 1	最少一次	At Least Once	1	消息保证至少送达一次，但可能重复（接收方需去重处理）	是	是	一般
QOS 2	正好一次(只有一次)	Exactly Once	2	消息保证只送达一次，既不丢失也不重复，开销最大	是	否	关键

3. 各等级QOS工作原理

(1)QoS 0 – 最多一次（At most once）

- 发送即忘（Fire and Forget）
- 消息发出后，不等待确认，也不重传。
- Broker 或 Subscriber 可能收到，也可能收不到，完全依赖网络状况。
- 优点：速度最快，资源消耗最低。
- 缺点：消息可能丢失，不保证可靠性。

- **典型场景**：传感器定时发送温度数据，偶尔丢一次也无所谓。

(2)QoS 1 - 至少一次 (At least once)

- 消息**保证至少送达一次**，但**可能重复**。
- 发送端 (Publisher) 发送消息后，会等待 Broker 返回一个 **PUBACK** 确认 (针对 PUBLISH 报文)。
- 如果没收到确认，发送端会**重发该消息**。
- 接收端可能多次收到同一条消息，因此**应用层通常需要自己做去重处理**。
- **优点**：消息不会丢失。
- **缺点**：可能收到重复消息，有一定额外开销。
- **典型场景**：设备状态上报、报警信息，允许少量重复但绝不能丢失。

(3)QoS 2 - 恰好一次 (Exactly once)

- 消息**保证只会被送达一次**，既不丢失，也不重复。
- 是 **最可靠但也是最复杂、最慢、开销最大** 的 QoS 等级。
- 使用 **四步握手过程** (**PUBLISH** → **PUBREC** → **PUBREL** → **PUBCOMP**) 来确保消息唯一性，避免重复和丢失。
- **优点**：绝对可靠，不多不少一次。
- **缺点**：协议交互复杂，延迟较高，性能开销大。
- **典型场景**：金融交易指令、设备控制命令等，要求严格一次执行的场景。

4.QOS传输

MQTT 的 QoS 实际上是**针对每一条 PUBLISH 消息在特定传输阶段所承诺的可靠性等级**，具体来说：

方向	说明
QoS between Publisher → Broker	指的是客户端 (Publisher) 向 Broker 发布消息时指定的 QoS 等级
QoS between Broker → Subscriber	指的是 Broker 向订阅了该主题的客户端 (Subscriber) 分发消息时使用的 QoS 等级

注意：

Publisher 发布消息时指定的 QoS，并不等于 Subscriber 一定能以相同 QoS 收到！

Broker 向 Subscriber 发送消息时，实际使用的 QoS 是以下三者中的 **最低值**：

- Publisher 发布时指定的 QoS
- Broker 自身对该主题消息的转发策略
- Subscriber 订阅该主题时声明的 **最大可接受 QoS (也就是订阅时指定的 QoS)**

例如：

- 如果 Publisher 用 QoS 2 发布，但 Subscriber 只订阅时声明 QoS 1，那么实际传递给 Subscriber 的是 QoS 1。
- 如果 Subscriber 订阅时声明 QoS 0，那么无论发布者用多高的 QoS，最终都按 QoS 0 处理。

5.QoS 与 SUBSCRIBE 的关系

当客户端使用 **SUBSCRIBE 报文** 订阅某个主题时，可以为每个主题指定一个最大可接受的 QoS 等级 (0/1/2) 。

Broker 在向该订阅者推送消息时，实际使用的 QoS 是 “发布者设置的 QoS” 与 “订阅者订阅时声明的 QoS” 两者的最小值。

举例：

角色	行为	QoS 设置
Publisher	发布主题 <code>sensor/temp</code> 消息，QoS = 2	高可靠性
Subscriber A	订阅 <code>sensor/temp</code> ，QoS = 1	最多接受 QoS 1
Subscriber B	订阅 <code>sensor/temp</code> ，QoS = 2	可接受 QoS 2

那么：

- Broker 发给 Subscriber A 的消息，实际使用 **QoS 1**
- Broker 发给 Subscriber B 的消息，实际使用 **QoS 2**

6.如何选择合适的QOS?

场景	推荐 QOS	原因
传感器定期上报温度/湿度等数据，偶尔丢一次无所谓	QOS0	省流量、省电，实时性要求不高
设备状态、报警信息，希望尽量不丢，允许少量重复	QOS1	平衡可靠性与性能，最常用
控制指令、配置更新、关键操作，必须准确执行一次	QOS2	保证可靠且不重复，但开销大
对实时性和可靠性要求都不高，且客户端处理能力有限	QOS0	减少处理负担
移动弱网环境，希望尽量可靠但能容忍一定延迟	QOS1	比 QoS 2 更高效，比 QoS 0 更可靠

7.QoS 与 Retained 消息 / 遗嘱消息 的关系

- **etained 消息（保留消息）**：可以指定 QoS，Broker 会以该 QoS 等级将保留消息推送给新订阅者。
- **Last Will（遗嘱消息）**：也可以指定 QoS，当客户端异常断开时，Broker 会以指定的 QoS 等级发布该遗嘱消息。

所以，这些特殊消息也遵循 MQTT 的 QoS 机制。

总结：MQTT 的 **QoS（服务质量等级）** 定义了消息从发布者到订阅者的传递可靠性，共有三个等级：**QoS 0（最多一次）、QoS 1（至少一次）、QoS 2（恰好一次）**，用户可根据业务对可靠性与性能的要求灵活选用。

七、MQTT 遗嘱机制

MQTT 的 **遗嘱机制（Last Will and Testament，简称 LWT）** 是该协议中一个非常有用的**客户端异常断线通知机制**，它允许客户端在连接到 MQTT 代理（Broker）时**预先设置一条“遗嘱消息”**，当该客户端**异常断开连接时（非正常主动断开）**，**Broker 会自动将这条遗嘱消息发布到指定的“遗嘱主题”上**，以通知其他客户端该设备或用户已经离线或发生异常。

1. 为什么需要MQTT遗嘱机制？

在 MQTT 的发布/订阅模型中，设备或客户端之间通常是**松耦合**的，一个设备（客户端）断开了连接，其他客户端**无法直接感知**。

但在很多实际应用中，我们往往需要知道某个设备是否在线，例如：

- 智能家居中，温湿度传感器突然离线，系统需要收到告警；
- 共享单车、物流设备掉线，后台需要及时发现异常；
- 一个关键服务或节点断开，其它服务需要做故障转移；

这时，**遗嘱机制就提供了一种简单有效的“客户端异常断线通知”方案。**

2. 遗嘱机制工作原理

(1) 客户端在连接时设置遗嘱信息

当一个 MQTT 客户端**连接到 Broker 时（发送 CONNECT 报文）**，可以在连接参数中指定以下与遗嘱相关的信息：

参数	说明
Will Topic（遗嘱主题）	当客户端异常断开时，Broker 会向该主题发布遗嘱消息
Will Payload（遗嘱消息内容）	遗嘱消息的实际内容，比如 <code>"设备离线"</code> 、 <code>"异常断开"</code> 等
Will QoS（遗嘱消息的 QoS）	指定遗嘱消息的 QoS 等级（0、1 或 2）

参数	说明
Will Retain（是否保留遗嘱消息）	是否将遗嘱消息设置为保留消息（Retained），新订阅者能立刻收到

注意：

这些遗嘱参数 **只有在客户端异常断开时才会生效**，如果是**客户端主动调用 DISCONNECT 断开连接**，则**不会触发遗嘱机制**

(2) 如果客户端异常断开，Broker 自动发布遗嘱消息

如果客户端由于以下原因**非正常断开连接**，例如：

- 网络中断
- 客户端崩溃
- 设备断电
- 没有发送 DISCONNECT 报文就断开了连接

并且 **在连接时设置了 Will Flag = 1（启用了遗嘱功能）**，那么 MQTT Broker 就会**自动向配置好的 Will Topic 发送一条 Will Payload 消息**，内容就是事先设置好的“遗嘱内容”。

这样，其他订阅了该遗嘱主题的客户端，就能立即感知到该设备已经离线或发生异常。

(3)如果客户端正常断开（发送了 DISCONNECT），则不会触发遗嘱

如果客户端在结束工作前，**主动发送了 DISCONNECT 报文**，正常断开连接，那么 Broker 会认为这是**预期内的断开**，**不会触发遗嘱机制**，也不会发布遗嘱消息。

3.如何在 CONNECT 报文中设置遗嘱机制？

在 MQTT 的 **CONNECT 报文（客户端连接请求）** 中，通过以下字段来配置遗嘱信息：

字段	说明
Will Flag（遗嘱标志位）	位于 CONNECT 报文中的 Connect Flags（连接标志位） 的第 5 位（从 0 开始数，第5位是 Bit 4） • 1：启用遗嘱功能 • 0：不启用
Will Topic	遗嘱消息要发布的主题（字符串，UTF-8）
Will Payload	遗嘱消息的内容（字符串或二进制数据）
Will QoS	遗嘱消息的服务质量等级（0、1 或 2）
Will Retain	是否将遗嘱消息设为保留消息（Retained）

这些信息是在 CONNECT 报文的 **可变报头（Variable Header）** 中的 **Connect Flags 与 Payload 部分** 指定的。

4.遗嘱机制使用示例：

场景：

一个温湿度传感器设备（ClientID = `sensor_001`）连接到 MQTT Broker，我们希望：

- 如果这个设备异常掉线（比如断电、网络中断），Broker 能自动向主题 `sensor/001/status` 发送一条遗嘱消息：`"sensor_001 is offline"`。
- 其他服务或设备订阅了 `sensor/001/status`，就能立刻知道该传感器离线了。

客户端连接时设置的遗嘱参数为：

参数	值
Will Flag	1（启用遗嘱）
Will Topic	<code>sensor/001/status</code>
Will Payload	<code>"sensor_001 is offline"</code>
Will QoS	1
Will Retain	1（设为保留消息，新订阅者能立即收到）

这样，一旦这个传感器异常断开，Broker 就会向 `sensor/001/status` 主题发送一条 QoS=1、Retained 的遗嘱消息，内容为 `"sensor_001 is offline"`。

5. 遗嘱机制应用场景：

应用场景	说明
智能家居设备监控	如传感器、智能开关等设备异常离线时，通知家庭网关或手机 App
工业设备监控	工控设备掉线时，通知运维系统进行告警或维修
共享设备 / 车联网	共享单车、车载终端等掉线时，后台系统检测异常
服务健康检测	微服务或边缘节点通过 MQTT 心跳/在线状态，异常时通知控制中心
集群节点状态通知	MQTT 用作集群通信时，节点宕机可通过遗嘱快速感知